

Предоставляю полную транскрибацию, разделенную в соответствии с названиями видеороликов, на основе видео на ютубе Григория Шаркова.

1. 5 ML Secrets I Wish I Knew Early

[Вступление/Общие рекомендации] five things I wish someone told me when I was learning machine learning Hey folks so I've been working as a machine learning engineer for the past 7 years And let me tell you there is a huge difference between what you learn online or in courses and what you will actually use on the job Today I want to share five things I really wish someone had told me back then when I was first studying machine learning I hope this helps you to avoid some of these mistakes And if you find this video helpful please subscribe because it really motivates me to do videos like this

Первое: Вы можете создать лучшую модель в мире, но все равно потерпеть неудачу First thing you can build the best model in the world but still fail I've seen people really brilliant ones building incredible models that hit the marks like 99% accuracy beautiful code and yet nothing gets deployed So no one uses it It just sits there forgotten in some notebooks Early in my career I was guilty of that as well I spent hours days and weeks tuning models chasing perfect statistics but could not even explain to a product manager what I was doing When I learned how to package my work and deploy this made it really useful to the others That's when the things click So if you're learning machine learning right now I'd say yes building great model is wonderful but don't stop there Learn how to package and deliver them to the real world

Второе: Не влюбляйтесь в свои метрики Second don't fall in love with your metrics I used to obsess over accuracy ROC AOC F1 squad you name it But no one outside the data team care And one day the product lead said "Cool but does this thing actually move the needle for us?" That's st But they were right You can have the best model in technically but if it isn't helpful to the business in some way or saving time making money or reducing risk is just academic So now I always ask myself is this thing actually useful how will it contribute to the company

Номер три: Поймите, как ваша компания зарабатывает деньги number three understand how your company makes money Seriously one of the biggest mindset shifts I had was when I stopped thinking like a data scientist and started thinking like a business person And if I was starting out again the first thing I would do is to try to understand how the company makes money where does the revenue come from what do they sell who is the customer what's the target audience and then ask myself how would a machine learning solution would actually make difference in that process I promise you once you see how your work ties into revenue cost or risk you will start build stuff that actually matters

Номер четыре: Как устроена бизнес-структура Number four how the business is structured Okay so now once you know how the company makes money the next thing I would do is to look at the organization chart and start figuring out how the pieces fit together who are the teams involved in delivering that value to the customer What are the bottlenecks what are

they trying to prove this helps you spot real opportunities where your skills can help instead of building a model in isolation and hoping someone finds a way to use it

Номер пять: Общение — это суперспособность Number five look I'm not saying you need to become a TED speaker but communication is a superpower I used to explain models using jargon and equations I really like that But then I saw that people eyes glaze over What helped me was a couple of good books on storytelling at sales and watching a couple of good videos on it and then just practice how to explain what I do without sounding like a statistical robot Now I focus on what problem are we solving why does it matter and how does a this solution help Applying that can already open so many doors So yeah the big mistake I see a lot of data scientists make isn't technical It's just staying stuck in the ML bubble But if you can understand the business focus on real world impact learn to explain things clearly and get your models into production you will be in a completely different league What has been the hardest part of your data science journey so far let me know in the comments I'd love to hear your stories

2. AI vs Scammers | How do you do that?

[Традиционное обнаружение мошенничества] have you ever had your bank suddenly block your card in the middle of a holiday or gotten a suspicious late night message asking if you really made this big purchase online it's annoying right but what's even scarier is when banks don't catch the fraud because fudsters today are faster and better equipped than ever before i know because I've been in this battle for over 15 years back in 2008 when I started working with data fighting fraud was pretty basic imagine Excel spreadsheets simple queries and basic rules we set up flags like an employee suddenly opening a dozens of miracles or authorize suspiciously large transfers each flag alone wasn't proof of fraud but together they told the story we got plenty of fraud that way but that was then fast forward to today you would expect banks to have evolved dramatically right but you might be surprised that many banks still use the same rulebased logic that we use it back in 2008 only now it's powered by SAS or Python script instead of Excel

[Слабость правил и угроза Генеративного ИИ] but there's a problem frosus have a ball and if you were a fza would you walk it straight into a trap knowing it's there of course not frosus adop using new technologies like generative AI deface and sophisticated global networks imagine a criminal using your child voice cloned by AI urgently asking you to send money can you send me some money i had an accident i really need your help sounds extreme right but it's happening right now and traditional rule-based fraud detection methods aren't built to handle that the core weakness of rulebased system is their rigidity they rely on predetermined criterias like transaction amount unusual locations and it's like setting traps in the forest expecting animals never to learn to avoid that over the time maintaining these rules becomes a headache costing banks money and irritating customers with false alerts eventually banks reach a point of diminishing returns more rules less accuracy and this is where AI enters the story not as a marketing buzzwo rd but as a genuine game changer

[ИИ как революционное изменение: Графовые нейронные сети (GNN)] what if banks could learn from frosters as quickly as frosters adapt think about a detective who never sleeps never forgets to face and can analyze billions of pieces of evidence in seconds that's the promise of AI one powerful AI approach gaining momentum is graph neural networks or GNS think of transactions not as isolated events but as a part of a massive network connecting accounts devices and locations gnn see the entire network they can quickly spot when multiple seemingly unrelated accounts are suddenly sending money to the same suspicious place they find fraud networks hiding in the plain sight major tech companies like AWS and Nvidia are already pushing this technology forward aws for instance uses GNNS to spend complex networks of transactions instant banks implementing this have significantly reduce fraud losses some by million dollars per year because model can catch fraud in real time something that traditional methods struggle to achieve

[Масштабирование и Контекст (GNN + LLM)] here is a catch it's easy to demonstrate a small cohort concept but running these advanced systems on scale of millions of customers and billions of transaction is an entirely different challenge it requires serious engineering robust infrastructure and specialized skills so what does an ideal fraud detection system look like in my view a perfect system doesn't just look at transactions it understands the context imagine each client or account has an evolving profile their risk score naturally rises not because of their actions but because of their connections you are who you associate with approach at scale combining graph neural networks with large language model is particularly promising here lms can parse huge amounts of unstructured data like emails chat logs images customer reviews and interaction adding yet another layer to of detection capability banks like GP Morgan Chase already use this to detect fraud signals hidden in everyday communications mastercard's generated API system for instance now analyzes trillions of data points catching fraudulent activities before they even happen they claim to have doubled their F detection rate while significantly reducing false alarms if you're thinking that sounds great why isn't every bank doing this the answer is complexity these technologies are powerful but they require significant investment and expertise to scale yet despite these challenges the shift towards AI driven solution is inevitable fraudes won't slow down so neither can we banks now face clear choice evolve with smarter adaptive systems or risk of falling behind in a rapidly changing fraud landscape what do you think have you experienced outdated fraud systems or have you seen banks effectively fighting back using AI share your thoughts below your opinion is very important if you found this helpful please like subscribe and stay tuned for more insights from the front lines of fraud detection until next time stay safe and vigilant

3. HOW I BECAME AN ML ENGINEER | THE PATH FROM JUNIOR TO SENIOR

[Вступление и Ранний опыт (Data Mining)] in this video I will share you my journey of becoming a machine learning engineer I'll tell you how I went from this to this I'll show you which

challenges I faced how I struggled on the job interviews but eventually ended up getting what I believe is the best job in the world and if you watch this video till the end I will show you how to get on track in studying machine learning and becoming better in this field I hope that my experience will encourage future machine learning Engineers when I started working in data science I had about 10 years of experience but it didn't help me much actually what I realized thanks to these 10 years is that I love working with data sometimes I mean no one called it data science back then but we used to call it data mining we were working with credit fraud data we were analyzing performance of individual loan Grant in officers whether they were any concentrations or fluctuation or any specific patterns in their behavior and the results were pretty impressive because at one point we discovered a credit loan renting scheme that consisted of many loans where some of the loans were used to repay previous loans so that that's why the whole portfolio was performing outstandingly well it was performing so well that the loan officer who was in charge of the schema was actually called an employee of the and several times another example we analyzed how traffic dropped in Paris region during a flooding period and this data we used to calculate how many backup workstations do we need in where to place them this is actually where I started using uh Coursera and DataCamp which I believe a very solid platform to start with if you're new to this field the problem was that uh the simple XEL was not enough for us to get the insights and to present those insights to the stakeholders in fact data is everywhere and if you know how to quickly collect combine and analyze your data you can collect all the low hanging fruits around you and show very impressive results

[Переход в ML и трудности поиска работы] as I said by the time I decided to turn to the dark side of the force and start working machine learning I had a senior position however in DS or ml world I was completely newbie so the job search took me about a year where I went to a dozen of interviews where I felt completely miserable I didn't know even half of the things people were asking me about people who interviewed me they seemed like real Guru Guru to me cuz they had a practical idea about the things that they were asking and they were asking about their all the day Fe teachers they were asking me about everything that they faced on daytoday life and it was so easy it seemed so easy for them but but I struggled with even with the simplest questions and they were asking me these questions and I just mumbled something incorrect and they rejected me one of the teams who rejected me they were doing multinomial classification which was completely new to me because for me classification was when you classify Cent items into two groups yes or no survived not survived however what they were doing they were putting clients into different into multiple bucket buckets depending on their risk level and of course after the interview they rejected me because I did not know what is multinomial logistic fession by the way this is not a very difficult topic so if you're interested let me know in the comments I'll do a separate video on that

[Успех через Коммуникацию и Обучение] 3 months later I saw that the same team was hiring again and I decided that if I could not impress them with the quality of what I know I can impress them with the quality of how fast I can learn and how well I can tell them the stories I quickly created a simple model that was predicting quality of wine depending on the its chemical structure how many alcohol how much how much alcohol how much sugar how much sulfites Etc you know in France Vine quality is very important of quality V and you know actually the model was very average but I made a presentation by showing which data did I which data I

used which how I built the model which results I achieved and what were the weak point and strong points of the models and how this model could be potentially improved in the future in this second interview I was not asked even a single question about the model everyone was very impressed by the difference of what they saw during the first interview and the second one or maybe they wanted to get my model as soon as possible they said they wanted to talk to other candidates and will come back to me uh in a week or something I was pretty much disappointed because I thought I failed again however to my great joy the next day they called me and said told me that I got this job two things I always tell to my colleagues first even an average model with good communication will outperform a the most complex and the most sophisticated model where the model Creator cannot clearly explain and communicate what this model is doing and why is it working second

[Советы по найму и успеху] d thing is that I now I've reached the level where I have hire people myself and very often I see people who came to this field just because they say that you can get some decent money here however if your eyes don't shine you don't progress that's that's why I always pay special attention to people who tend to who are trying to switch their career when hiring them because from what I have observed very often these people come there they have good soft skills but they lack some technical expertise which means that with proper guidance they will progress at an Norm pace and over 6 months or one year they will outperform you with the task that you thought you are good at my tips to for Success first strengthen your fundamental skills in statistics math and probability personally for me a statistics and data science program uh from MIT was a huge help I would definitely recommend it to anyone new in the field second start working on on a specific project think of any idea that you might h ave that you want to implement and start implementing it thir next is keep consistent effort there are a lot of people who have learned something about machine learning however there are much less much fewer people who can who understand how it works and how to apply it in the real life next is practice first in the interviews you can quickly understand whether the person the answers that the candidate is giving you are just the answers that he'd learned from YouTube or they he really fa those issues and he really understand how to how to handle and how to address different things take as a fact that the data will be messy and noisy in most of the things that you learn in during when taking courses the data especially prepared for you and there's a conventional solution to how this task should be addressed however in the real life because of the noise or because of the imperfection of the data it will not be like this this is absolutely normal this is a part of the job over the time yo u will learn how to address those issues with enough by keeping your C curiosity level high enough next accept the fact that the data will never be perfect it will always be messy and noisy but an imperfect solution is always better than a nons solution which means that if you have a solution which could be approved your more experienced colleagues will guide you through this especially at the more Junior rules so just keep on working and iteratively improving Your solution and the last one is the mo is the most important takeaway for me to transition to machine learning engineer hole is understanding that any solution that we make as data scientists or machine learning Engineers are actually part of the it we are writing programs the only difference with classical it programs is that part of these programs have not been explicitly written by you but got trained based on the data that you have have however it Still Remains a program that has to be deployed so if you

want to succeed in machine learning and ml engineering role you will have to invest some time into software engineering skills because um there are two types of models and the ones that are in Productions are the programs in production so all the laws all the procedures and all the good practices and all the bad practices as well are applicable in this field you will be amazed by how much you can improve if you start applying them following those six tips you will make it so if you found this video interesting or helpful don't forget to like And subscribe thanks see you soon

4. HOW IS TEAMWORK organized in Data Science PROJECTS?

[Введение: Теория против Практики и Коммуникация] when we study machine learn we'll often focus on how to clean data train models valuate their performance but there's one very important step that is missing here it's actually the teamwork so let's talk about how you can Implement those teamwork practices in your projects and what should you do to become a valuable team member when working on real life projects here is the input raw very often messy data get fed into some magic box and this magic box this is where the all the the magic happens all the analysis deployment model training and then results of this magic are consumed and deployed to the production to serve happy users easy right but that's the practice what's the difference between theory and practice in theory they are the same in reality they're not the reality of T workk at University or during online courses we are often taught how to work with data and how to create models out of this data but uh the teamwork skills remain at a very basic level and it's not surprising because teamwork skills are very hard to train during the online courses but without them you will not survive as data scientists we often like to focus on breathtaking new algorithms or new model architectures that we want to implement as soon as possible but very few people like writing clean code writing unit test or documented the whole thing and moreover communication with business users becomes pretty hardest especially towards the end of the project trust me poor communication has killed more projects than bad mons in fact bad Bens often are resulted by poor communication for example there is a large number of prerequisite steps you have to do as a team before jumping into the modeling F you need to understand the problem understand the date and the process that generated it validate and talk through with the users about all the anomalies That You observe in the data and understand how the model is going to fit into the existing it infrastructure remember garbage in garbage out so make sure you plan enough time for these steps before you start trying new fancy modeling outbreak and this whole process actually requires a lot of communication with the business users we tend to skip those steps before saying they are too technical so that the users will have hard time understanding it but actually focusing on communication during the the early stage of the project will allow you to build a very important currency it's the most important currency in the whole project actually it's the confidence between business users and you try putting yourself at the pace of the business users they have a problem they invited some very smart people to work on this program these people come to them give them the black box and they have to appropriate results produced by this black box and as a business users you want

to be confident that these guys they understand you they understand your problem and they understand how the solution that you are bringing to them will give them real value

[Качество кода и Pull Requests (PR)] full requests remember you are participating in a software project so whatever code you write will be used by you and other team members for many many times that's why the code quality is a must and that's why pull requests are essential to ensure the code quality but working with pull request is not about smashing the approved button it's an opportunity for you and your team members to have a fresh pair of eyes looking at the code that they have generated and seeing that the code quality and the architectural decisions that have been taken they make sense and they can bring real value to the basement of the project first of all it's what mentioning that pull request is a great tool to enforce coding standards in your team of course for simple code formatting things you can use tools like black or flake8 that will auto format your code but there are things beyond that for example the naming convention how you structure your files etc and this also can be standardized across the team and that's why during pull request pay attention to this on the other hand if your code is being reviewed by someone else make sure that you don't take it personally the person who is reviewing the code has made a very important effort to understand what you are doing and to propose a solution that might be more effective so it's not about saying you wrote the bad code it's about saying hey let's make our code better and lastly very important thing is that don't be shy about asking questions when we are writing the code we have a very clear logic of why we are writing it the way we do it but it might not be obvious for the other people so these questions will help to understand the way you were thinking and to challenge it or probably improve it in the future remember pull request is not about catching all the bugs in the code a pull request is about code quality so the more you do them the stronger your team becomes in creating reusable and maintainable code that can be further put in production

[Переход от Jupiter Notebooks к Python Files и повторное использование кода]

transitioning from Jupiter notebooks to python files Jupiter notebook is a great tool for quick prototyping and experimentation so when it comes to real world projects your code has to be modular and when you are writing this code make sure that you don't write it only for yourself for example do not focus on writing the code that trains the model but rather focus on writing a pipeline that trains a given type of a model with given parameters in that way the code will be reusable by the other members of the team when they will want to train their own model from my experience the code repositories are very often separated into three main parts the engine engineering model training and evaluation and inference code reusability python has a very rich set of libraries that probably solve one of the problems that you are currently facing so please make sure you do not reinvent the wheel try searching for these packages and see whether you can use them in your project for specific repeating function put them into separate utility functions when it comes to practice for example I have set of functions that travels with me from one project to another and these functions cover wide variety of features from data pre-processing to model training hyperparameter tuning model evaluation model registry and so on and so forth these functions are pretty generic but the more I use them the more generic they become when it comes to inference make sure that your model is not just a model object but it's rather a cyclic learning pipeline that includes as many pre-processing steps as possible into this object pythonic learning pipelines is a great way of automating these pre-processing steps and

making sure that your model can handle the unexpected data correctly another aspect of code reusability is dependency management so far I've been widely using virtual environments and cond environments for my projects but the rule is always one project one environment recently we're thinki ng about switching to poetry if you have any experience or feedback about poetry please let me know in the comments

[Юнит-тесты] un need tests right unit test yes even for data pre-processing as I said before you are participating in a software engineering project that's why when you are contributing to the work of the others first of all you want to make sure that your code does not break the others code and vice versa when the others will be modifying sampling they want to make sure that they don't break what you have done that's why unit test is a very important as usually this adds from 30 to 50% of additional time required for the develop but actually faster development does not mean faster typing faster development means lower debugging time and Immunity test serve that purpose they can drastically decrease the amount of time you will spend debugging your and someone else's C another very important indicator related to unit test is their complex the go up down says that if you have to write a very complex test to test your function there is a way to simplify your function and decouple components so that you can test them separately and independently and this will improve the overall architecture of your code

[Управление данными и хранение моделей] when you're working on large projects you need to make sure that you've agreed on how the data will distracted in your project where do you store the input data where do you store the output data where do you store intermediary data and how do you version it moreover usually I also like trying to freeze the input data especially the one that's coming from the production system the thing is that the Productions data has its own life cycle it gets created updated and remove According to some rules that you might not be aware of that's why the data that you have been using 6 months ago you may not be able to reproduce it in exactly the same way that's why you may please make sure that when you're working with go production data you freeze it at least for the modeling site second very important aspect is that the data in the storage should not be modified manually and directly everything should be put into ETL pipelines that allow you to load and transform data and store it somewhere else with a new state and make sure that someone wants to modify it let's say add a couple of more months to the training data it's not done by directly but rather the pipeline roome that produ this training data is modified the PO request gets approved and then make sure that you have agreed on who and when will run this by by so that the other's work is not affect model storage in order to put one more name to production you and your colleague will probably have to rerun the whole modeling pipeline over and over again more than 100 times That means that whenever the pipeline is run the results and the outputs are stored correctly in the model registry ml4 is the most common model registry but there are other model Registries on the market that you can use for your purposes the idea of storing these versions is the following not all of your experiments will be successful for example you want to make sure that you are able to easily come back to the Modi that you had three days ago without necessity to rerun the whole modate ply because it might take too much time that's why whenever someone runs the mod pipeline its result have to be stored along with the predefined metrics that you and your team have decided that you will be focusing on this further allows a simpler way of choosing between the different follows

[Автоматизация развертывания (CI/CD)] automating deployment or cicd pipes it's unlikely that you will build the ideal model from the first round so what ideally you would want to do is to set up an iterated incremental process of improvvy so you start with an MPP and then gradually you start adding additional features and additional capabilities to your modeling project and during this process you want to make sure that you will not only deliver the code but you can also deploy it in the environment so whenever someone merges a p request into the main branch or to the dep Branch you want to make sure that this code is tested is checked for errors and this package and is the point at P to the development by me usually development process is done done manually requires a lot of manual operations of putting Heap files here moving them there and so on and so forth remember that your goal is to deliver new features to the users quickly and reliably when working with C PLS you have a chance to automate all these things any manual interventions during the deployment process are your enemies so make sure you don't leave this technical Gap behind you some of these skills may not be very easy to find but you you can develop them as a team and work on them as a team to improve your team's performance remember your ultimate goal is to deliver a software project to the business then this project has to bring real value and be integrated into the existing it infrastructure so st art small incrementally increase the number of features that are available in your product participate in code reviews make sure that the code that you produce is of a good quality and test it and that the data and the models and the code are properly ver if you like that video please smash that like And subscribe button and uh see you next year

5. Junior to Senior ML Dev: Level Up Without Burning Out

[Введение и распространенные ошибки новичков] are you stuck fixing bugs in someone else's code or overwhelmed by endless tutorials well you're not alone today I'll show you exactly what separates juniors from seniors in machine learning and how to bridge that gap without burning out let's turn your frustration into a road map first you're trying to learn every algorithm for 90% of the cases assuming that your task is well defined and the data is well prepared in proving that you will get from switching from XG boost to cat boost will be marginal instead focus on one algorithm for every family and this will be a good solitizing code over communication you are creating a product for business so talking to business is more important than 100% test coverage I usually spend around 60% of my time talking two people and only 40% and this is if I'm very lucky doing coding third ignoring the why behind the models it has happened a couple of times in my career that we leaked the answers to the model and the model performance was pretty dece nt but we did not bother too much to investigate why in fact answering those why question is the most productive thing you can do to your model why am I organizing training data like this or there is another way or why am I under sampling my training data like this or there is another way juniors often drawn in 100hour courses but seniors usually stack skills strategically

[Стратегическое накопление навыков (Base, Mid, Top Layers)] and here is how first base lag required for survival python G and two three key algorithm like regression trees neural

network for example build a good enough model first even if the A is 0.7 mid layer debugging pipelines data storytelling and basic cloud deployment creating the whole end to end pipeline is what senior capable of so if you want to progress toward that field you need to learn that skill top player leadership system design mentoring cost to wear and for example cloud cost versus accuracy Skip mastering SVM camels focus on skill that let you ship projects not just notebooks when the model fails juniors w

[Когда модель выходит из строя: Фокус на данных] ill usually focus on tweaking hyperparameters for days while seniors will ask "Is the data rotten or did we misdefine the problem?" Usually this will bring a much bigger impact to your model's performance here's the senior's checklist for this point validate your data distributions make sure that train and test split distributions are similar check that evaluation matrix are aligned with business goals for example precision precall for fraud detection and thirdly simplify replace neural network with logistic regression and probably performance will be similar so next time your model crashes start with the data not the code

[Привычки Senior-специалиста] seniors don't work harder they work smarter so here are some useful habits for you to develop first is the 80/20 rule spend 80% of your time for 20% of the task that really matter for example data cleaning versus tuning second set boundaries no coding after 7:00 p.m well usually seniors protect their focus time that's the most precious resource that you can have in t he project third learn to say no you don't need to join every meeting protect your deep work hours juniors will think leadership is management seniors will know it's about influence try this first try to mentor someone explain a concept to an intern and you will master it faster from my personal experience you understand any concept if and only if you are able to explain it in simple and structured way second document your decisions write a one paragraph in a markdown file explaining why for every model choice that you make seniors will try to anticipate those questions over the time as you become senior you will start anticipating those questions third speak to business translate accuracy improved by 2% into the model says \$500 per year for example a junior shared a model rock curve but senior said this reduces our customer chunk by 15% guess who gets promoted

[План действий (Cheat Sheet)] are you ready to level up here's your cheat sheet quarter one build three end to- end project simple is okay put them on GitHu b quarter two volunteer to debug a teammate's code learn from their mistakes if you have a chance to review a pull request never miss this opportunity quarter three lead a tiny project for example automate data validation or model data collection or model monitoring quarter four teach a 5-minut concept at a team meeting well you need to accept that none of us has been born an expert we become seniors by practicing in public remember seniors were once juniors who kept trying focus on skill that matter protect your energy and share your achievements i will suggest you to do these three things right after you finished watching this video first delete unused Corsera tubs pick the project to finish ask a senior what is one thing that I should stop doing bookmark this video to watch in 6 months to track the growth and don't forget to like subscribe and stay tuned for

6. NEURAL NETWORKS VS. MAFIA! 100\$ WILL TELL US EVERYTHING ABOUT YOU

[Проблема Контекста в Обнаружении Мошенничества] I've got a very simple question for you does it matter whether I send this \$100 to my grandmother or I send it to the most notorious criminals in my city actually this is the question you are going to face when you will be dealing with fraud detection and it's a difficult question because the frauders what they will do they will try to mimic real world Behavior that's why this \$100 you're about to send to your grandmother will have exactly the same properties as \$100 some will try to send to Don Carone the same goes for electronic transactions from from the bank's point of view they will be just two independent roles in the transaction data set here's the thing in front detection context is everything and your connections help to discover a lot of information about your real context and your real behavior in this video I will show you how graph neural network can help you to discover those patterns and how this \$100 may not be as innocent as it seems imagine you discovered that a middle-aged woman calls a middle-aged man from times to times it seems innocent right and the same man helps to run a local business to another young man still looks pretty innocent but then you discover that the same man is connected to Walter White and Gustavo Fring who are the most notorious criminals of the city and the last two don't go well with each other in this moment you start to realize that this network is not that obvious and there might be some connections that you are not aware of because of the context and it actually comes true you discover a lot of episodes where those guys are connected with each other so as I said your context defines who you are you can't fake it and you can't ignore it if you want to build robust fraud detection systems

[Используемые данные и ограничения классического ML] so for the purpose of this video I will be using leptic Bitcoin transaction data set that is available on kago I will put the link in the description it is actually a synthetic data set which means it does not have the True Values but the reason I chose it is that this synthetic data was generated using the real data so it mimics the real world Behavior as in real life your data will contain known bad transactions known not bad transactions and a huge number of transactions whose status is unknown usually people will put those unknown transactions into the group of known not bads but actually it's a very strong assumption that you have to check and you have to think carefully before taking this decision in our case around half of the transactions are known illicit transactions around 20% are known elicited transactions and the remaining 30% are unknown status transactions in real life every client and every transaction will be represented by a list of features characterizing it think about it some client age amount of the transaction total client turnover country that the client belongs to etc etc in our case since we're operating with the synthetic data this data will not be meaningful for us but they will have a similar distribution think about it as if someone who wanted to make this data set Anonymous ran a PCA algorithm on these features and the features that you see in this data set represent results of this PCA transformation so as I said our goal here is to build a model that will be able to learn from these features and the connections between different elements in the data set and help us to predict illicit transactions and before we jump into details let's talk about why graph neural networks is

the goto Choice when you work on this type of projects in the classical machine learning approach we would take each transaction as an independent observation build a feature Vector for each transaction and we will hope that a machine learning algorithm will be able to find those patterns that distinguish legit transactions from illicit but in the transaction world no one is isolated so the fact that my context depends on the context of people that I'm connected to really helps graph neural ne

[Почему Графовые нейронные сети (GNN) работают] works to find and discover much more complex patterns to discover fraudulent activity think about it as in convolution neural networks when the value of a point does not only depend on the value itself but also depends on the value of the points surrounding it graph convolution layer will work exactly the same so at the first layer it will allow the node not only access its own context but also allow to get the context from its direct neighbors as we add more layers we add possibility for the gcnn not only to learn from local context but also to learn from the global context surrounding each node if you want to understand more about how graph convolutions work I highly recommend this article on tensorflow tensorflow and pytorch have packages that help you to deal with graph neural networks and if we were to build a system for production my personal choice would be tensorflow but but for the sake of this tutorial I will be using pytorch because it's a bit easier to set up a bit easier to train and a bit easier to explain how it's working so in this video we've explored the business problem identifying fraudulent transactions understood the data the Bitcoin elliptic data set and understood the method that we're going to use the graph convolutional neural networks in the next video we will build a minimal viable product of a graph convolutional neural network and build a model that is capable of predicting depending on the context how innocent the \$100 that you are about to send are in the banking world so what do you think about graph neural networks would you like to try them are you excited about them let me know in the comments please like subscribe and hit the Bell button so that you don't miss the next video when it gets published

7. RAG: Stop LLM Hallucinations & Ditch the "Black Box" (Finally, some control)

[Введение и Проблема (Галлюцинации)] how rack technology changes the way we work with large language models Have you faced a situation like this: You ask ChatGPT something simple and straightforward and the answer is complete nonsense that looks legit? hi I'm Gregory your MLOps navigator and for the past 3 years I've been helping my clients to implement generative AI in their business process and you know what I've noticed the biggest problem of modern large language models is that they often provide you with facts that don't exist. According to Stanford AI ChatGPT's responses contain factual errors in every third answer! but this leads to a leap in confidence that causes companies to slow down adoption of AI mckenzie recently reported that 57% of their clients consider this problem as the major factor stopping them from going allin with AI especially when it comes to the most valuable business process

That's why for professionals working with LLMs this is a major issue that goes far beyond theoretical discussion. Hallucinations is the most common and precise term that describes this phenomenon, when a model is confidently generating factually inconsistent information, like citations of non-existent research papers or alternative historical facts. I remember when I first faced the after GPT3 got published I was working on a project using a specific code coding framework and I was trying to add some functionality that was not supported by it after several hours of searching over the internet I asked ChatGPT and it provided me with an answer that looked elegant and simple that's just the way I was expecting it to work that was cool there was one problem it didn't work What GPT actually did is that it simply translated my text prompt into the code that look legitimate in the context of the framework and that's it. It didn't know how the framework works, which features does it provide and how they are supposed to be used? When you are generating a cool-looking essay or working on a sci-fi novel this might look funny but in medicine, finance or legal areas this is unacceptable You can't automate your business processes that affect your life, money or security using a tool which is that noisy. So this is where the rack or retrieval augmented generations comes onto the scene.

[Традиционный подход vs. Подход RAG] The traditional approach for large language text generation models is simple: in the traditional approach for a large language text generation that would be pretty straightforward you ask a question and the model uses its weight which actually represent the knowledge it obtained during training to generate you the answer but these weights can be inaccurate, outdated or incomplete facts RAG approaches differently there are two main steps the first one is retrieval and search and second is generation so when a user asks a question the system first searches the external database and finds the relevant pieces of information then adds them to the system prompt and after that sends the whole prompt to the model along with the original question this approach forces a model to answer not only using its internal memory but also using irrelevant up-to-date facts.

[Компоненты RAG] Any RAG system has the following components: first one is the data storage this is a database of documents used to form the context this context could be documents research papers internal database any information you like even your client internal data. This is a database of documents used to form the context. This could be documents, research papers or internal databases (let's say your client data) second is the index the input data storage is split into smaller chunks and passed through an embedding model the model that is used to translate the meaning of these chunks into a format that machines can understand which is numbers third one is the search engine the search engine transforms the user query into the same language used by the index and finds which chunks are the most relevant given the user query. The LLM. It takes a system prompt (which consists of the user query and the relevant chunks combined) and generates the final answer.

[Примеры и преимущества RAG] And voila this approach drastically increases reliability of the LLM answers. Financial institutions can now create AI application that help their relationship managers to analyze how they help their clients to grow their business or manage risks. Medical clinics can use GenAI to help doctors to analyse the patient's symptoms and suggest treatment. Researchers can now analyze vast amount of research data opening the path to new discoveries and these are not experiments these are realworld cases that exist today It is hard

to underestimate the advantages that you get when using RAG the first one of course is the factual correctness the system works with the sources internal or external that you trust. Relevance. The system works with upto-date information, with the cut-off date defined by you, not by the model training date. Transparency. The system is able to provide you with citations to support the claims it is making.

[Практика: Настройка и зависимости] Now let's move to the funniest part. Let's create a simple RAG system. Here is a list of components that I chose to start with. I have to mention that this is a rapidly changing ecosystem and every day there are new tools that come onto the market. You do not need to learn how to use them today by heart. Just try to understand the main concept so that when you face this problem and you have to build your own rag system you know which elements you need to succeed. Here are the main components we're going to use for this demo: the first one is the lang chain we're going to use lang chain to orchestrate all the components this is the most popular and common package used today on the market. FAISS for vector storage. I chose it because it is in-memory. For production cases I would really suggest you using something more serious like pine cone or neoforj. OpenAI API for embeddings and code generation. In fact you can use any LLM available today on the market: OpenAI, Claude, Gemini, Deepseek etc. Let's start by installing dependency we will need lang chain itself lang chain openai to ensure that we can use openai API inside the lang chain and lang chain community which is a package which is developed by the langchain community and contains a handful of different classes and tools that we're going to use throughout this tutorial next we're going to use python.env ENV package to be able to securely store and retrieve API components without necessity to commit them inside the code and the face CPU package that is a package that will allow us to create the vector store so you can install them using pip install or any other dependency management software that you are using then uh once we have installed them we will need to import them we will need some standard libraries like globe JSON OS and uh typing any for type annotations we will need to load the loadenv function that will allow us to load our credentials from the env files and then we have a handful of lang chain imports if you want just take a pause and copy them for yourself and of course the link to the notebook will be available in the description and the final step that we're going to need is to set up some constants so we'll need to load open AI key from the env file if you don't have an open AI key it's super easy to get just go to openai.com uh API sections and create a token for yourself then we're going to specify which the path to the folder that contains the documents that we're going to use for the rag then we've got the chunking parameters which is the chunk size and the chunk overlap i'll talk about them later and we'll specify which embedding model and which LLM model we're going to use these are just string constants that we'll supply to langgraph. For this particular case we will teach the LLM to answer questions about the recent news about 5 companies: Amazon, Apple, Google and Microsoft.

[Загрузка и нарезка данных (Chunking)] for loading we're going to be using something very simple basically we'll iterate in the we will define we will find all the txt files located in the documents folder and then using the lang chain text loader we will load this every every single document and then we'll also add some metadata for each of these documents metadata represents some information that is not directly the contents but rather the context of a file that is used for the rag. Once it is done, we have a list of Documents. Document is a special class

available in lang_chain community package. It holds metadata and content of each news item. OK, now that we have loaded all the documents, we can chunk them into pieces. why do we need to do that that's very simple in general a single news item can be pretty large therefore we will need to split it into smaller pieces but when doing so we need to make sure that we keep some artifacts and restore connections between the chunks that's where we add chunk overlap part to the system chunk_size and chunk_overlap are the two hyperparameters that you would want to play with when you are tuning your RAG system the shorter the chunks the smaller and more specialized chunks you're going to get and the bigger the chunk overlap will affect the number of chunks that will be generated out of your docu data. For chunking we will use the RecursiveCharacterTextSplitter. We pass our chunk_size and chunk_overlap parameters as well as we can specify separators as a list of patterns. in order to chunk the documents we'll be using chunk_documents function which receives a list of documents and the parameters that will be used to create these chunks so we'll be using recursive character text splitter which is the splitter available in the lang chain community we will pass it with the chunking parameters and we'll also specify the patterns that are going to be used to create these chunks because it's a text splitter you can say stop at 1,000th symbol or you can say stop when you reach closest to the 1,000 symbol but for example don't split the words by half and this is what we are doing here when specifying the separators so in order to create the chunks we will call the method split_documents and pass it the list of the documents that we loaded at the previous stage and as in many cases don't forget about the metadata that you want to associate with each chunk because once the chunk is split you need to store some metadata that will point to the original source that the chunk has been created so in our case we will be adding chunk ID chunk_size and the total number of chunks so let's print some stats uh and see what we have so we split our 15 documents into 80 chunks each chunk is something between 145 and 997 characters and uh different companies produce different number of chunks that's because the difference in the length of the news items and each chunk example will look something like this so it will have some chunk text as well as the metadata that inherited partially from the documents and was added during the chunking part so when we talk a

[Векторизация и Векторное хранилище] About vectorization this is very simple embeddings is a numerical vector that represent the meaning of the phrase. I am using OpenAIEmbeddings, but this is an example. All modern LLM support this operation so you can play around with different ones. so for this sake of this tutorial we'll be using open AI embeddings we will need to pass the name of the embedding model and the API key that we want to use and then call the method embed_query and pass it the text and receive the output. so let's create a function that will create embeddings for the different chunks so in this case we'll take page content of each chunk and create embeddings the way I described above by calling the embed_documents and we'll return a list of embeddings for each chunk now we have all the components to create our vector database the vector database in general will have two main components which is the text of an item in our case it's a chunk and a vector representation that will be used to search for this text this architecture allows to create a very fast searching mechanism because searching numbers is much faster and more efficient than searching textual data. we take a text of a query as an input, we pass it through the same LLM and receive the embedding that will be then fed into the vector store. The vector storage will retrieve K chunks that are most similar to the user

query. for this we'll be using the create face vector store which will take a list of documents and the embedding model so in order to create the vector database we are going to need the list of content of each page and we'll also need to provide it with the list of metadata and turning these page contents into embeddings will be handled by face automatically so we'll need to supply it with the embedding model that we want to use for this one so if we take a look at this we created a vector saw consisting of 80 documents it has 80 vectors and each vector has the same dimension that we described before. now that we've created the vector store we can simply send queries to this database and say what are the closest chunks that correspond to the user query so for example if we ask about what is the latest news about Amazon what we will need we will need to call a method which is called similarity search with score we'll provided the text of our question and we'll say that we want no more than two items to be fetched for us.

[Извлечение данных (Retrieval) и Генерация промпта] now we are ready to create rag retriever class responsibility of the rag retriever will be to find the chunks that are relevant to a user query and format them so that they can be injected into the system prompt. This class will have 3 methods: 1. Retrieve documents, based on a user query 2. Retrieve documents with scores (a debugging method that not only will retrieve chunks, but also will report similarity scores for each chunk) 3. Get_context_string. A method that will generate a string containing the text of all selected chunks. now we're ready to generate the rack prompt let's take a look at the system template which is the instruction that we are going to give to an LLM which will not depend on the user query so we are saying that you are helpful AI assistant that answers questions based on the provided context your task is to analyze provided context carefully answer the user question using the information from the context be accurate and site your sources when possible if the context does not contain the enough information say so clearly and if you're unsure about something acknowledge the uncertainty and then we have put a placeholder for the context that will be retrieved based on the user query second part of the uh system prompt will be the human template which is straightforward a placeholder for the question and then we provide a an ask to the model to provide a detailed answer based on the context above so the generate rag prompt will be a very straightforward function it will take a string question and the rag retriever that we're going to use and it will create a chat prompt template which is the template for any chat template inside the la ng chain where we can define different messages that are sent to the lang chain model and in our case we'll provide the system template as a system message and the human template as a human one.

[Генерация финального ответа и сравнение] so now we are ready to create a generate answer function this will be the function that will collect all the pieces we've built so far into one place so basically this function will take a question we'll take an LLM model and we'll take the retriever which is the class that we created above first given a user prompt and the retriever we can generate a rag prompt we can add the question to the to this prompt and invoke the LLM with the formatted message once we have it we can now return a dictionary that will contain the original question the the response as well as the citations for the sources let's add the file company and the similarity score. Let's quickly check the answers from an LLM without RAG. For that I prepared another system prompt that does not contain the latest news and hence responds only with the data, stored inside the LLM. Now you can see the difference, don't you?.

[Переход в продакшн и Заключение] As I said before, the tools I am showing will probably get obsolete in the next couple of years, but the main approach will remain the same. Even today there is a handful of tools available for scaling this system for example you can add support for different chunking options code and images should be transformed differently from the simple text or you can add support for more advanced vector search database or for example you can add support for knowledge graph generation your documents contain or even turn this whole system into a multi-agentic application that use more specialized agents that will use large language models to retrieve challenge cross check information before it is being sent to the and of course don't forget about the computation time optimization different chunks can be processed in parallel faster models can be used more optimal solutions for caching can be applied as well etc etc. The technologies are secondary. The primary goal is to understand how the system's been built and how it works under the hood. Let's wrap up. RAG is not a technological improvement. It is a fundamental shift in the approach that you use Large Language Models. It solves one of the most critical problems – unreliable data. But implementing RAG will require careful planning: Choosing the relevant and reliable sources. Deciding how and when the information will be updated. Choosing and tuning hyper parameters. In exchange you get a significant improvement of quality and reliability of the answers that you get from an LLM.

8. WHAT ABOUT AI VS ML

[Восприятие ИИ] if I ask you to think about AI what image would come into your mind some people will think about some humanoid like robots from black Miro series or some people will think about Marvel Jarvis AI assistant of the Iron Man or some people will think about hundreds of thousands of line millions of lines of code passing through your PC but today what's happening is that more and more people will think about AI as a chat gpt or genini any chat board and we've got handful of them around us but what if I tell you that the science fiction movies are actually are much closer to the original definition of artificial intelligence and the chat gpt is just a text generation model that acts like a human but is not actually artificial intellig what is AI what is the difference between Ai and machine learning and where is the border line let's talk about this in this video

[Компьютерная Программа vs. Машинное Обучение (ML)] let's start with a simple computer program actually it's just a list of instructions of how to take input data transforming According to some internal logic and then produce the output if the logic is not too complex or can be defined can be known in advance then just make such a tree of those eanl statements would be the best CA of course you can apply some fancy software engineering patterns in the end it will still be a list of eanl statements and this behavior is known well in advance before the program is put into production what if we don't know exactly every single rule that we have to apply to achieve the result and we have a historical set of data that we can use to learn to derite those roles for us in this case we can use machine learning algorithm basically in this case we will feed previously known data to the the algorithm and it will derive all the conditions and all

the tree decision trees for us and it's possible that there will not be only one decision tree that might be or so many of them however the final result will be defined as an interpretation of those decision trees that have been trained using the input data in this case we don't specifically say what are the conditions or what is the protocol we want the program to follow we the only thing that we can Define is the hyperparameters of the machine learning algorithm we can tell it which features should it use which can tell it which Target variable if there is a target variable should it use which met metric should it use when it trains and so on and so forth if we correctly set up all the parameters we fed in the right training data and we fed in the right target variable though the algorithm will train it for us but what if we have relatively structured data I don't know some text or some tables or some images but we don't know which exact features should be used to solve a problem how they should be transformed of of course there's always an option to try out a bunch of combinations and see which work best but eventually we will reach some limit either we will be limited by time or computational cost or just patience

[Нейронные Сети (NN)] but what if we don't know which exact features should be used in order to solve our problem I mean our input is relatively structured we know what we want to achieve but we don't know which features should we take how to process uh those features in this case you will need to use neural networks the more I use them the more I admire the elegance of this solution and this year we can finally celebrate that two people who made fundamental contribution to development of neural networks got their Nobel Prize in physics even though they did not create chbt odali but without their fundamental discoveries none of them would be possible John hopfield created the concept of associated memory without going into much details the way it works is inspired by human brain and that makes it very flexible and allows storing and restoring very complex information from a generalized versions of it that's the way how human remember things right because we don't remember every single details about what has happened in the past but rather we have some flashlights in our brain that allow us to restore what has happened in the past jofrey Hinton made a significant contribution to the back propagation algorithms a nutshell it works in a very similar way like human brain works when it's training and this method allows multi-layer net neural networks to train and learn from the data that is fed into them in this case we can no long Define which features the model should be using but the only thing we can Define is the number and the characteristics of connections between different neurons which transform the input data to the desired output during the training the neural network will yield a sample from the training data will make predictions we'll check by how much it has missed and depending on the how much it has missed it will try to update all the parameters of the network to minimize the error in the future and this process will continue over and over again until the model breaches the

[Заключение: Определение ИИ] state with the least possible error going back to the original question what is AI and where is the difference with machine learning I can say that from my point of view neural networks today are the closest we have ever been to the real artificial intelligence however a program that is not able to understand that there is a world outside the boundaries of the problem so they can be super good in solving one specific task but they will not be able to realize that this task may be useless they are not able to realize that more other more important tasks that have to be solved that's why people in Sci-Fi movies talking about Skynet or Jarvis or other very smart Technologies they're much closer to the actual definition of

AI but however in modern world of course uh a lot of people will call chat GPT and AI whereas it is a simple Text generation program anyways thanks a lot for watching and hope you enjoy this video please don't forget to like And subscribe

9. What's the #1 MLOps mistake costing careers in 2026? (And the skills to fix it)

[Введение и Самая большая ошибка] (0:00 - 0:05) [MUSIC: Upbeat, modern tech intro music fades in and out] (0:00 - 0:05) [GRAPHICS: ML Ops Navigator Logo with "Grigory Sharkov" subtly beneath, animating in and out. Clean, professional aesthetic.] (0:05 - 0:10) [MEDIUM SHOT] Grigory: Hey everyone, Grigory Sharkov here, your ML Ops Navigator. (0:10 - 0:25) [VISUAL: ANIMATION: A perfectly trained ML model's graph/code snippet in a pristine digital notebook on screen. Slowly, dust particles start accumulating digitally, then a cracked screen overlay. Simultaneously, B-ROLL: stock footage of a bustling office, then a frustrated person at a computer.] (0:10 - 0:25) [PACING: Starts calm, builds tension slightly.] Grigory: Imagine your meticulously built ML model, stuck. Never reaching production, never delivering value, just gathering digital dust in a forgotten notebook. Or worse, it *does* make it to production, only to fail spectacularly, costing your company millions and your career priceless reputation. This isn't a hypothetical nightmare; it's the number one career trap for MLOps Engineers in 2026.

[Рост рынка и Экспертиза] (0:25 - 0:35) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "MLOps Market: >\$19.55 Billion by 2032" (Bold, prominent). ANIMATION: A growing bar chart or infographic representing market growth.] (0:25 - 0:35) [PACING: Energetic and informative.] Grigory: The MLOps market is exploding, projected to exceed an astounding \$19.55 billion by 2032. But the skills required are evolving just as fast, especially with the rise of Generative AI. Without the right, *production-proven* skills, you risk being left behind, unable to build systems that truly work and deliver business impact. (0:35 - 0:55) [CLOSE-UP] [LIGHTING: Slightly more direct, emphasizing Grigory's credibility.] [BACKGROUND: Professional, blurred office or tech setup.] Grigory: I'm Grigory Sharkov, an ML and AI engineer with 7+ years of experience building and deploying real-world, production-ready systems. I've seen firsthand what breaks and what makes an MLOps engineer invaluable. Today, I'm sharing the critical skills you *must* master to navigate this rapidly changing landscape and ensure your models – and your career – thrive.

[Понимание Уровней Решений (Quick Win, Systematic, Overkill)] (0:55 - 0:57) [TRANSITION: Quick, sharp cut to a clean title card.] --- (0:57 - 1:00) [GRAPHICS: Full screen title card: "Understanding Solution Tiers"] (1:00 - 1:20) [MEDIUM SHOT] [BACKGROUND: Clean, simple backdrop.] Grigory: Now, for each crucial skill we'll discuss, I won't just tell you *what* to learn. I'll give you a practical roadmap on *how* to implement it, tailored to different stages of your MLOps journey or project maturity. For every skill, you'll find three distinct solution options: (1:20 - 1:30) [VISUAL: TEXT OVERLAY: "Quick Win: Simple, Immediate

Improvement"] [ANIMATION: Small, rapidly expanding green checkmark icon.] Grigory: * Quick Win: These are simple, immediate actions or tools that can offer basic improvement. Perfect for getting started, small projects, or rapid prototyping when you need to show value fast. (1:30 - 1:40) [VISUAL: TEXT OVERLAY: "Systematic Solution: Robust, Scalable Standard"] [ANIMATION: A gear icon turning smoothly.] Grigory: * Systematic Solution: This represents a more robust, standard, and scalable approach or set of tools. It's what most production-ready MLOps teams strive for, providing a solid foundation for reliable, efficient, and maintainable systems. (3:20 - 3:55) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Overkill Solution: Full MLOps Platform, Immutable Data Lakes, A/B Testing, Disaster Recovery"] [GRAPHICS: Complex MLOps platform visualization, Kubeflow/TFX logos, Delta Lake/LakeFS logos, A/B test split graphic, world map with redundant servers glowing.] Grigory: * Now, for the overkill solution with extra contingency: implement a fully automated MLOps platform like Kubeflow Pipelines or TFX, complete with immutable data lakes using technologies like Delta Lake or LakeFS for robust data governance. Utilize advanced canary deployments and comprehensive A/B testing frameworks for seamless rollouts and rollbacks. And, critically, include multi-region disaster recovery strategies for your ML services to ensure ultimate resilience. This approach maximizes uptime, drastically reduces risk, and ensures business continuity even under extreme conditions. (1:40 - 1:55) [VISUAL: TEXT OVERLAY: "Overkill Solution with Extra Contingency: Advanced Resilience"] [ANIMATION: A shield icon with multiple layers, implying protection.] Grigory: * Overkill Solution with Extra Contingency: These are advanced, highly resilient, and comprehensive solutions, often including redundancies, cutting-edge practices, or multi-faceted strategies for ultimate robustness. These are for mission-critical systems where failure is simply not an option, or when you need to optimize every last ounce of performance and reliability. (1:55 - 2:05) [VISUAL: GRAPHICS: Simple infographic illustrating the progression from "Quick Win" (small, simple block) to "Systematic" (medium, interconnected blocks) to "Overkill" (large, complex, redundant blocks) with increasing complexity and robustness. Use arrows to show progression.] (1:55 - 2:05) [MEDIUM SHOT, looking at graphic then back at camera.] Grigory: Your choice among these tiers will depend on your project's size, budget constraints, your team's maturity, and most importantly, the criticality of the ML system you're building. Let's dive into the skills! (3:55 - 4:10) [CLOSE-UP] [Grigory looking thoughtful, then gesturing to emphasize the point.] Grigory: The MLOps tool landscape is vast and constantly evolving. Don't get bogged down trying to learn every single tool. Focus on understanding the underlying principles – robust automation and versioning – because these fundamentals remain constant, even as specific tools change.

1. Beyond CI/CD: Building Resilient ML Pipelines that *Never* Fail (2:05 - 2:07)

[TRANSITION: Fast wipe to next section title card.] --- ## (2:07 - 2:10) [GRAPHICS: Section Title Card: "1. Beyond CI/CD: Building Resilient ML Pipelines that *Never* Fail"] (2:10 - 2:20) [MEDIUM SHOT] [VISUAL: ANIMATION: Animated flowchart showing a broken, then fixed, ML pipeline with various MLOps components like versioning, testing, monitoring appearing as Grigory speaks.] Grigory: First up: building resilient ML pipelines that simply *never* fail. This means mastering the core MLOps principles of robust automation, continuous integration and continuous delivery specifically for ML, thorough data and model versioning, automated testing, continuous data quality monitoring, and ensuring full reproducibility across the entire ML

lifecycle. (2:20 - 2:35) [CLOSE-UP] [PACING: Slightly more emphatic.] Grigory: Why is this critical? Because this set of skills prevents models from being perpetually stuck in development. It effectively addresses critical issues like data drift and concept drift, ensuring stable, consistent, and predictable model performance in production environments. It's the absolute foundation for avoiding technical debt and those unexpected, costly failures I mentioned earlier. (2:35 - 2:37) [GRAPHICS: Text overlay: "Solution Options: Resilient ML Pipelines" (Briefly flash on screen)] (2:37 - 2:40) [MEDIUM SHOT] Grigory: Let's look at how you can implement this, from quick wins to ultimate resilience. (2:40 - 2:55) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Quick Win: Basic Versioning & Testing"] [SCREEN CAPTURE: Simple Git commands, a mock `README.md` showing dataset version.] Grigory: * For a quick win, start with basic Git for code versioning. Manually track dataset versions for small projects—maybe a simple README file with dataset hashes. Implement simple unit tests for your model's core logic. This offers immediate control over your codebase and is a first step towards reproducibility. (2:55 - 3:20) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Systematic Solution: Automated CI/CD, DVC, MLflow, Evidently AI"] [GRAPHICS: Flowchart animation showing CI/CD pipeline, DVC logo, MLflow logo, Evidently AI logo appearing as Grigory names them. Highlight key integrations.] Grigory: * A more systematic solution involves establishing automated CI/CD pipelines, perhaps using GitHub Actions or GitLab CI, for code validation, testing, and deployment. Integrate dedicated data versioning tools like DVC and model registries such as MLflow for comprehensive version control. Implement automated data quality checks, using tools like Evidently AI, and build comprehensive model evaluation tests directly into your pipeline. This provides a standardized, repeatable process for reliable model deployment. (3:20 - 3:55) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Overkill Solution: Full MLOps Platform, Immutable Data Lakes, A/B Testing, Disaster Recovery"] [GRAPHICS: Complex MLOps platform visualization, Kubeflow/TFX logos, Delta Lake/LakeFS logos, A/B test split graphic, world map with redundant servers glowing.] Grigory: * Now, for the overkill solution with extra contingency: implement a fully automated MLOps platform like Kubeflow Pipelines or TFX, complete with immutable data lakes using technologies like Delta Lake or LakeFS for robust data governance. Utilize advanced canary deployments and comprehensive A/B testing frameworks for seamless rollouts and rollbacks. And, critically, include multi-region disaster recovery strategies for your ML services to ensure ultimate resilience. This approach maximizes uptime, drastically reduces risk, and ensures business continuity even under extreme conditions.

2. Cloud-Native Power: Architecting Scalable & Secure MLOps Infrastructures (4:10 - 4:12) [TRANSITION: Fast wipe to next section title card.] --- ## (4:12 - 4:15) [GRAPHICS: Section Title Card: "2. Cloud-Native Power: Architecting Scalable & Secure MLOps Infrastructures"] (4:15 - 4:30) [MEDIUM SHOT] [VISUAL: ANIMATION: Illustrating scaling up resources (CPU/GPU icons multiplying), secure data flow (encrypted tunnel graphic), and different cloud platform logos (AWS, GCP, Azure).] Grigory: Next, we talk about Cloud-Native Power: architecting scalable and secure MLOps infrastructures. This means proficiency in major cloud platforms, a deep understanding of containerization and orchestration technologies, and the ability to implement Infrastructure as Code, or IaC, to manage your MLOps environments efficiently. (4:30 - 4:45) [CLOSE-UP] [PACING: Confident, slightly faster.] Grigory: Why is this

critical? These skills enable the deployment of ML models at true scale, allowing for efficient management of computational resources like CPUs and GPUs. More importantly, they allow you to implement robust security protocols and compliance measures for sensitive data in production. It's about moving your experiments from a local machine to making a global impact. (4:45 - 4:47) [GRAPHICS: Text overlay: "Solution Options: Scalable & Secure MLOps" (Briefly flash on screen)] (4:47 - 4:50) [MEDIUM SHOT] Grigory: Here are your solution options for cloud-native MLOps: (4:50 - 5:05) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Quick Win: Manual VM Deployment, Basic Docker"] [SCREEN CAPTURE: Simple `docker build` command, a basic cloud VM console view.] Grigory: * For a quick win, manually deploy ML models to a single virtual machine in a cloud environment. Use basic Docker containers to package your model's dependencies. This gets your model into a production-like setting quickly to validate initial functionality and learn the cloud basics. (5:05 - 5:35) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Systematic Solution: Managed ML Services, Kubernetes, IaC"] [GRAPHICS: AWS SageMaker, GCP Vertex AI, Azure ML logos. Docker whale icon, Kubernetes logo, Terraform/CloudFormation logos. Animated visual of services flowing together.] Grigory: * A systematic solution involves leveraging managed ML services on a major cloud provider, such as AWS SageMaker, GCP Vertex AI, or Azure ML, to streamline operations. Continue utilizing Docker for packaging and local development, and introduce basic Kubernetes deployments for orchestration of your containerized ML workloads. Implement Infrastructure as Code with Terraform or CloudFormation for provisioning core infrastructure resources. This provides scalable and manageable infrastructure, reducing manual effort and ensuring consistency. (5:35 - 6:10) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Overkill Solution: Multi-Cloud, Advanced Kubernetes, Service Mesh, Policy Enforcement"] [GRAPHICS: Hybrid cloud icon, multiple cloud logos, complex Kubernetes cluster diagram, Istio logo, OPA logo, security scanner icon. Dynamic, interwoven network illustration.] Grigory: * For an overkill solution with extra contingency, you'd design and implement multi-cloud or hybrid-cloud MLOps infrastructures for maximum flexibility and to avoid vendor lock-in. Employ advanced Kubernetes deployments with a service mesh like Istio for fine-grained traffic management, enhanced security, and observability. Integrate sophisticated IaC with policy enforcement, using tools like OPA, and include automated security scanning directly into your deployment pipelines. This ensures extreme scalability, high availability, and cutting-edge security for mission-critical ML applications. (6:10 - 6:25) [CLOSE-UP] [Grigory gestures to a blank space as if it's a map or a diagram.] Grigory: While it's great to know about all platforms, mastering all of them is unrealistic. Focus on understanding cloud-agnostic concepts, and then gain deep proficiency in at least one major cloud provider's MLOps ecosystem.

3. LLMops & Generative AI: Operationalizing the Next Wave of Intelligent Systems (6:25 - 6:27) [TRANSITION: Fast wipe to next section title card.] --- ## (6:27 - 6:30) [GRAPHICS: Section Title Card: "3. LLMops & Generative AI: Operationalizing the Next Wave of Intelligent Systems"] (6:30 - 6:45) [MEDIUM SHOT] [VISUAL: ANIMATION: An LLM responding to prompts (text bubbles), then showing monitoring dashboards with LLM-specific metrics (e.g., perplexity, safety scores) and a drift detection graph.] Grigory: Now, let's talk about the game-changer: LLMops and Generative AI. This is about specific MLOps practices tailored for the unique challenges of Generative AI and Large Language Models. This includes efficient fine-tuning

workflows, serving LLMs at scale, real-time monitoring of LLM outputs for quality, safety, and drift, advanced prompt engineering in production, and managing the distinct types of model and concept drift that are unique to generative models. (6:45 - 7:00) [CLOSE-UP] [PACING: Urgent, emphasizes importance.] Grigory: This is critical because the rise of Generative AI is reshaping the entire ML landscape. MLOps engineers who can effectively operationalize these complex models will be at the forefront of AI innovation, ensuring these powerful models deliver value reliably and safely in production. Ignoring this trend could lead to rapid skill obsolescence, so pay close attention. (7:00 - 7:02) [GRAPHICS: Text overlay: "Solution Options: LLMops & Generative AI" (Briefly flash on screen)] (7:02 - 7:05) [MEDIUM SHOT] Grigory: Here are your options for operationalizing LLMs: (7:05 - 7:20) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Quick Win: Fine-tune small LLMs, Simple API Deployments, Manual Checks"] [SCREEN CAPTURE: Python script showing a Hugging Face model fine-tuning snippet, a simple API request/response.] Grigory: * For a quick win, experiment with fine-tuning smaller open-source LLMs on consumer GPUs. Deploy these models via simple API endpoints. Conduct manual, qualitative checks of the generated outputs. This allows rapid iteration and exploration of Generative AI capabilities without a huge investment. (7:20 - 7:50) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Systematic Solution: Managed LLM Platforms, Prompt/Dataset Versioning, Automated Metrics"] [GRAPHICS: Hugging Face, OpenAI, Vertex AI logos. Version control icon over text/data, ROUGE/BLEU metrics chart, embedding drift graph.] Grigory: * A systematic solution involves utilizing managed LLM platforms, like Hugging Face Inference Endpoints, OpenAI API, or cloud LLM services such as Vertex AI for LLMs, for efficient serving and scalability. Implement version control for your prompts and fine-tuning datasets. Develop automated quantitative evaluation metrics, like ROUGE, BLEU, or custom similarity scores, and basic drift detection for embeddings or output distributions. This enables reliable and measurable deployment of LLMs, with a strong focus on consistent performance. (7:50 - 8:30) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Overkill Solution: Custom LLM Serving, Adversarial Testing, RLHF, AI Governance"] [GRAPHICS: NVIDIA Triton logo, quantization animation. "Adversarial Testing" graphic with a broken prompt. RLHF loop diagram. "AI Governance" flowchart.] Grigory: * For an overkill solution, you'd build custom LLM serving infrastructure optimized for extreme scale and low latency, perhaps using NVIDIA Triton Inference Server or custom quantization techniques. Develop advanced adversarial testing for safety, bias, and jailbreaking attempts. Integrate Reinforcement Learning from Human Feedback, or RLHF, into continuous improvement loops, and establish comprehensive governance frameworks for model behavior and ethical guidelines. This provides maximum control, robust safety measures, and continuous self-improvement for critical generative AI systems. (8:30 - 8:45) [CLOSE-UP] [Grigory holds up a generic "adaptability" icon or flexes his hands.] Grigory: This is a rapidly evolving field. Your focus here should be on developing a flexible mindset for adapting to new advancements, understanding the specific challenges of LLMs, and applying MLOps principles to these new paradigms, rather than expecting a fixed set of solutions.

4. The MLOps Translator: Bridging Tech & Business to Deliver Real Value (8:45 - 8:47)

[TRANSITION: Fast wipe to next section title card.] --- ## (8:47 - 8:50) [GRAPHICS: Section Title Card: "4. The MLOps Translator: Bridging Tech & Business to Deliver Real Value"] (8:50 -

9:05) [MEDIUM SHOT] [VISUAL: ANIMATION: Three animated figures representing a Data Scientist, Engineer, and Business Stakeholder, initially separated, then a bridge forming between them as Grigory speaks.] Grigory: Finally, and perhaps most crucially, we have "The MLOps Translator": bridging tech and business to deliver real value. This means developing strong soft skills, including effective communication and collaboration with data scientists, software engineers, and business stakeholders. It encompasses understanding specific business objectives, translating technical work into tangible Return on Investment, or ROI, and navigating the complex ethical and governance considerations inherent in AI systems. (9:05 - 9:20) [CLOSE-UP] [PACING: Slower, more thoughtful and empathetic.] Grigory: Why is this critical? MLOps isn't just about technical execution; it's about enabling ML models to deliver meaningful business value. Without these crucial "human" skills, even technically perfect models can fail to gain adoption or achieve their intended impact, becoming just another piece of digital dust. This directly addresses that crucial "bridging the gap" challenge we often see in the community. (9:20 - 9:22) [GRAPHICS: Text overlay: "Solution Options: The MLOps Translator" (Briefly flash on screen)] (9:22 - 9:25) [MEDIUM SHOT] Grigory: Let's look at how to become that essential translator: (9:25 - 9:40) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Quick Win: Informal Check-ins, Simple Documentation"] [GRAPHICS: Two speech bubbles connecting, a simple README file icon.] Grigory: * As a quick win, schedule regular informal check-ins with your data scientists and engineers. Document the business problem a model is solving in a simple README file. This fosters basic alignment and understanding across your immediate team members. (9:40 - 10:10) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Systematic Solution: Cross-functional Groups, Business-focused Reports, Proactive Communication"] [GRAPHICS: Interlocking gears icon, a business report with charts, a megaphone icon for proactive communication.] Grigory: * For a systematic approach, establish cross-functional MLOps working groups that include data scientists, software engineers, and key business stakeholders. Create clear, concise reports that translate model performance and technical metrics into tangible business impact—think ROI, cost savings, or user engagement. Proactively communicate potential risks, limitations, and ethical considerations of AI systems. This builds stronger bridges between teams, ensuring MLOps efforts align with strategic business goals. (10:10 - 10:50) [MEDIUM SHOT] [VISUAL: TEXT OVERLAY: "Overkill Solution: MLOps Product/Program Roles, AI Governance, Explainability/Fairness Tools"] [GRAPHICS: "Product Manager" title over a figure, "AI Governance Committee" badge, SHAP/LIME logos, fairness metric charts. Complex, multi-layered visual representing governance and transparency.] Grigory: * For the overkill solution with extra contingency, implement dedicated MLOps product management or program management roles to oversee the entire ML lifecycle from a business value perspective. Establish formal AI governance committees and develop advanced AI ethics frameworks. Integrate tooling for model explainability, such as SHAP or LIME, and fairness analysis directly into your continuous validation pipelines, making transparency and responsible AI a core part of the MLOps process. This ensures that ML initiatives are not only technically robust but also strategically aligned, ethically sound, and fully compliant, driving maximum long-term value. (10:50 - 11:05) [CLOSE-UP] [Grigory points to his head, then makes a "bridge" gesture with his hands.] Grigory: Soft skills are often harder to quantify and "learn" than technical ones, but they are just as important. Focus on practical tips,

real-world examples, and fostering a collaborative mindset. Your ability to communicate will make or break your MLOps initiatives.

[Заключение и Призыв к действию] (11:05 - 11:07) [TRANSITION: Smooth dissolve to end card.] --- (11:07 - 11:15) [WIDE SHOT] [VISUAL: ML Ops Navigator End Card with social media handles, prominent subscribe button animation, and a space for Grigory.] (11:07 - 11:15) [MUSIC: Outro music begins, upbeat and inspiring.] Grigory: So there you have it: the top skills for an MLOps Engineer in 2026. Master these, and you won't just keep up; you'll lead. (11:15 - 11:20) [VISUAL: TEXT OVERLAY: "Even the best performing model is worth nothing until put into production." (This is the channel's specific call to action style.) Prominently displayed.] Grigory: Remember, even the best performing model is worth nothing until put into production. Master these skills, and you won't just deploy models; you'll build intelligent systems that truly work and deliver real impact. (11:20 - 11:35) [MEDIUM SHOT] [VISUAL: Grigory gestures to the comments section below, then to the subscribe button.] Grigory: What are your biggest MLOps challenges right now? Share them in the comments below! Don't forget to like this video and subscribe to ML Ops Navigator for more insights into building production-ready ML systems. (11:35 - 11:40) [OUTRO MUSIC & GRAPHICS: ML Ops Navigator logo, social handles, subscribe button animation. Music fades out.]

10. (Вставленный текст: GenAI Chatbot Evaluation Roadmap)

Main message Your GenAI chatbot demo was a hit in the POC, but why isn't it in production? The answer lies in evaluation, and it's far more complex than you think. Many data and machine learning engineers get excited about the potential of generative AI, building impressive proofs-of-concept. However, when it comes to deploying these solutions into the real world, projects often stall due to concerns around cost, inconsistent performance, and a lack of proper governance and observability.

In this video, I'll cut through the hype and show you the practical steps to move your generative AI agents from fascinating demos to robust, production-ready systems that deliver real business value. With my 7+ years of experience building production-ready ML and AI systems, I'll share actionable insights on how to rigorously evaluate and confidently deploy these complex agents.

Roadmap 1. Beyond Traditional Metrics: Understanding Generative AI Agent Evaluation

The "Worst Mistake": Treating generative AI agent evaluation like traditional ML model evaluation (e.g., just accuracy for a classifier). Generative agents are non-deterministic, multi-step, and tool-using, demanding a new approach.

- **Why it's Different:**
 - Non-deterministic Nature: LLMs can produce varied outputs for the same input, making reproducibility and consistent evaluation a challenge.

- Tool Usage & Multi-step Reasoning: Agents interact with external tools and perform complex, chained operations. Evaluating tool selection, parameter accuracy, and handling failures is crucial.
- Context and Memory: Agents need to maintain conversational context and memory, impacting performance over longer interactions.
- **Key Production-Ready Metrics to Focus On (Quick Win):**
 - Task Success/Completion Rate: Did the agent achieve its ultimate goal? (End-to-end evaluation)
 - Tool Utilization Accuracy: Was the correct tool chosen, and were its parameters used correctly?
 - Robustness: How well does the agent perform under varied or noisy inputs, edge cases, and adversarial attacks?
 - Safety & Alignment: Does the agent avoid harmful outputs, biases, or prompt injections?
 - Cost Efficiency & Latency: Monitoring inference costs and response times is critical for production scalability.

2. Architecting for Observability and Debugging in AgentOps

- **The Challenge of Debugging:** Non-deterministic, multi-component agent workflows are notoriously hard to debug. A simple error can be hidden deep within a chain of LLM calls and tool uses.
- **Systematic Solution: Implementing Agent-Centric Observability:**
 - End-to-End Tracing: Logging every step of an agent's reasoning path, including LLM calls, tool interactions, and intermediate thoughts. Tools like LangChain's tracing, MLflow, and custom logging are vital.
 - Structured Logging: Capturing critical information (inputs, outputs, latency, token usage, tool status, error messages) at each step to pinpoint failures.
 - Real-time Monitoring Dashboards: Visualizing agent performance, error rates, token usage, and latency in production to detect issues proactively.
 - Identifying Failure Points (Practical Example): How to use traces to diagnose a tool failure, a hallucination, or an incorrect reasoning step in a multi-turn conversation.

3. Building a Continuous Agent Evaluation Pipeline with GenAIOps

- **Systematic Solution: Integrating Evaluation into CI/CD:** Continuous evaluation is paramount for agents, as their performance can drift due to model updates, prompt changes, or shifts in user behavior.
- **Automated Evaluation Strategies:**
 - LLM-as-a-Judge: Using a more capable LLM to evaluate the agent's output against predefined criteria, task success, or factual correctness. This allows for scalable, automated testing.
 - Code-based Evaluators: Writing custom code to check specific aspects, such as JSON output format, presence of keywords, or database query correctness.

- Synthetic Data Generation: Creating diverse test cases, including edge cases and adversarial prompts, to rigorously test agent robustness.
 - **Human-in-the-Loop (HITL) for Overkill & Contingency:**
 - Why HITL is Still Crucial: For subjective tasks, high-stakes applications, and verifying automated evaluations, human review is indispensable.
 - Integrating Human Feedback: Designing UIs for human annotators, collecting feedback on specific agent turns, and using this data to retrain or refine agent prompts.
 - Red Teaming & Adversarial Testing: Actively trying to "break" the agent by injecting malicious prompts or unexpected inputs to identify vulnerabilities.
 - Tools and Frameworks: Mention frameworks like LangChain, DeepEval, RAGAS, MLflow, and Vertex AI for building these pipelines.
 - **Weaknesses and Constraints**
 - Complexity & Resources: Building a comprehensive GenAIOps evaluation pipeline requires significant engineering effort and can be resource-intensive, especially for smaller teams without dedicated MLOps expertise.
 - Lack of Standardization: The generative AI space is evolving rapidly, leading to a lack of universally adopted evaluation metrics and tools, which can complicate comparisons and best practices.
 - The "Perfect" Evaluation Myth: Due to the inherent non-determinism and vast output space of LLMs, achieving 100% perfect, exhaustive evaluation is practically impossible. The goal is robust, continuous improvement, not static perfection.
 - Not a Silver Bullet for Bad Design: Even the best evaluation won't save an agent that is fundamentally poorly designed or lacks clear objectives.
 - **Call-to-Action:** "Even the best performing model is worth nothing until put into production. Master agent evaluation and turn your generative AI dreams into real-world solutions. Share your biggest pain points in getting generative AI agents into production in the comments below!"
-

11. (Вставленный текст: Outline: The Evolving Data Science Identity: Why Your Role is Splitting (and How to Adapt for 2026))

Main Message Is your data science career future-proof? The skills you need for 2026 are changing faster than ever. The traditional "data scientist" role is specializing, splitting into focused paths like ML Engineer, Data Analyst, and the rapidly emerging Analytics Engineer. If you're a data engineer or machine learning engineer, this shift presents both challenges and massive opportunities to build production-ready systems that truly work. Even the best performing model is worth nothing until put into production – and that's where your expertise becomes non-negotiable. I'm Grigory Sharkov, an ML and AI engineer with 7+ years of

experience building these very systems, and I'm here to share the real-world insights you need to navigate this evolution and thrive.

Roadmap 1. The MLOps Imperative: Building Production-Ready ML Systems

- **Key Point:** MLOps is no longer a buzzword; it's the backbone of successful machine learning in production, addressing critical challenges like model deployment, monitoring, and scaling.
- **Solution:**
 - Step 1. Quick Win: Implement foundational MLOps practices immediately: version control for models and data (DVC), basic model serving with Docker, and simple monitoring for model drift using open-source tools.
 - Step 2. Systematic Solution: Establish CI/CD pipelines for ML, robust monitoring and alerting for model performance and data quality, and embrace reproducibility across the ML lifecycle. Tools like MLflow, Kubeflow, or TFX become central.
 - Step 3. Overkill Solution with Extra Contingency: Architect a fully automated, scalable MLOps platform integrated with cloud services (AWS SageMaker, GCP Vertex AI, Azure ML), incorporating advanced governance, cost attribution, and security measures. Focus on proactive drift detection and automated retraining.
- **Weaknesses and Constraints:** Initial MLOps setup can be resource-intensive and requires a shift in mindset towards software engineering rigor. Not all organizations have the maturity or budget for a full-scale MLOps platform, making step 1 and 2 more practical for many.

2. Data Engineering's New Frontier: Fueling AI with Robust Pipelines

- **Key Point:** As AI models become more sophisticated, the demand for scalable, real-time, and high-quality data pipelines built by data engineers is exploding, especially for Generative AI.
- **Solution:**
 - Step 1. Quick Win: Master Python and SQL as foundational skills. Focus on building reliable batch ETL pipelines using tools like Apache Airflow or Dagster.
 - Step 2. Systematic Solution: Dive into big data technologies like Apache Spark and streaming platforms like Kafka or Kinesis for real-time data processing. Embrace cloud-native data services (e.g., Google BigQuery, Snowflake, Databricks Lakehouse) and infrastructure as code for pipeline management.
 - Step 3. Overkill Solution with Extra Contingency: Design and implement advanced data observability frameworks, self-healing pipelines, and multi-cloud data strategies, ensuring extreme data quality and governance, particularly for sensitive data used in AI applications.
- **Weaknesses and Constraints:** The sheer volume and velocity of modern data can overwhelm traditional data engineering approaches. Integrating diverse data sources and ensuring data consistency across complex systems remains a significant challenge. Learning multiple big data tools can have a steep learning curve.

3. Navigating Generative AI: From Experimentation to Production for Engineers

- **Key Point:** Generative AI isn't just for data scientists; ML and Data Engineers are crucial for operationalizing these models, managing their unique data requirements, and addressing ethical concerns.
- **Solution:**
 - Step 1. Quick Win: Understand the fundamentals of generative AI models (LLMs, Diffusion Models) and how they consume and produce data. Experiment with open-source frameworks like Hugging Face or LangChain.
 - Step 2. Systematic Solution: Focus on building efficient data pipelines for training and fine-tuning generative models, addressing data privacy, bias detection, and ethical deployment. Learn how to optimize inference for these computationally intensive models.
 - Step 3. Overkill Solution with Extra Contingency: Develop strategies for prompt engineering at scale, implement robust guardrails and content moderation for generative outputs, and explore federated learning or differential privacy techniques for sensitive data in GenAI applications.
- **Weaknesses and Constraints:** Generative AI models often have high computational costs and can suffer from "hallucinations" or biased outputs, requiring careful monitoring and mitigation. The ethical and regulatory landscape for GenAI is rapidly evolving, making long-term solutions challenging to predict.

4. Future-Proofing Your Career: The Power of Adaptability & Business Acumen

- **Key Point:** While technical skills are essential, soft skills like business acumen, communication, and continuous adaptability are becoming non-negotiable for success and career longevity in the evolving data landscape.
- **Solution:**
 - Step 1. Quick Win: Actively seek to understand the business context of your ML/data projects. Ask "why" are we building this, and "what problem does it solve?" Practice explaining technical concepts in simple terms to non-technical stakeholders.
 - Step 2. Systematic Solution: Cultivate a "product sense" by actively participating in requirement gathering and understanding user needs. Prioritize projects with clear business value. Continuously engage with industry trends and new technologies, dedicating time to learn beyond your immediate project needs.
 - Step 3. Overkill Solution with Extra Contingency: Become a technical leader who not only builds but also influences strategy. Mentor junior team members, lead cross-functional initiatives, and proactively identify opportunities where data and ML can drive significant business impact.
- **Weaknesses and Constraints:** Developing strong soft skills takes time and consistent effort; it's not a switch you can flip. In highly specialized roles, it can be easy to lose sight of the broader business context if not actively pursued. Some organizational cultures may not fully value or facilitate the development of these skills.

12. (Вставленный текст: Proficient Prompt Engineering for Agentic Applications: Outline)

Title What is the worst nightmare for ML Engineers building agentic applications? And how to prevent it with Proficient Prompt Engineering.

Main Message (Hook) Imagine spending weeks building an intelligent agent, only for it to fall apart in production – acting inconsistently, hallucinating critical information, or worse, becoming vulnerable to malicious inputs. This isn't just a hypothetical problem; it's a very real nightmare for ML engineers trying to move beyond theoretical LLM performance to production-ready agentic applications. (Why care) The truth is, the best performing model is nothing until it's reliably in production. Agentic AI promises incredible automation and intelligence, but without proficient prompt engineering, these systems become brittle, unreliable, and fail to deliver on their immense potential. You need strategies that transform your LLMs into robust, autonomous agents that can plan, execute, and adapt in the real world. (Why trust me) With 7+ years of experience building production-ready ML and AI systems, I've seen firsthand the challenges of getting these complex systems to "just work." I've transitioned from business to tech, learning what it truly takes to build scalable and reliable data and ML pipelines. Today, I'm sharing battle-tested strategies to help you navigate the complexities of agentic prompt engineering, ensuring your AI agents don't just perform well in testing, but thrive in production.

Roadmap 1. Introduction: Beyond Basic Prompts – The Agentic Shift We'll kick things off by defining what truly sets agentic prompt engineering apart from basic LLM prompting. It's about designing an architecture, not just a query.

- Understanding Agentic Design Principles: The fundamental shift from simple queries to architectural thinking for autonomous AI.
- Defining the Agent's Persona & Goals: How to craft powerful system prompts that clearly establish the agent's role, objectives, desired output format (e.g., JSON), and constraints from the outset.
- The "Driver's Seat" Mentality: Why you need to actively guide your agent's behavior, emphasizing clear, positive instructions rather than vague directives.
- Single Responsibility Principle for Agents: Breaking down complex tasks into manageable sub-agents or functions, mirroring good software engineering practices.

2. Core Agentic Prompting Techniques: Architecting Intelligence This section focuses on the practical techniques that empower your agents to think, plan, and act reliably.

- Chain-of-Thought (CoT) and Self-Reflection: Guiding the agent to "think step by step" to improve accuracy and enable self-correction by evaluating its own outputs before acting.

- Prompt Chaining for Complex Workflows: Breaking down multi-step tasks into a sequence of smaller, manageable prompts, where the output of one feeds into the next, enhancing control and interpretability.
- Tool Use Engineering: How to design clear, unambiguous tool definitions and specifications, ensuring your agents can effectively interact with external systems and APIs.

3. Structured Prompt Engineering: Building Robust Blueprints Moving beyond ad-hoc prompting, structured prompt engineering provides a systematic approach to building robust and predictable AI agents. It's about creating a blueprint for your agent's interactions.

- **What is Structured Prompt Engineering?**
 - Defining it as a methodology that uses a predefined framework or template to design prompts, ensuring consistency, clarity, and higher performance in LLM systems.
 - Emphasize its role in reducing off-topic responses and inconsistent formatting by guiding the LLM towards relevant information.
- **Key Components of a Structured Prompt:**
 - Role (Persona): Clearly defining the AI's identity and perspective (e.g., "You are a seasoned VC," "You are a helpful travel assistant").
 - Task/Instruction: The specific action the agent needs to perform, stated with precision (e.g., "Create a detailed itinerary," "Classify the text").
 - Context: Background information crucial for the task, including audience specifics, nuances, or previous interactions (e.g., "Considering the increasing importance of sustainability," "From the following meeting notes").
 - Objective: The ultimate goal or desired outcome of the output.
 - Constraints: Rules or boundaries for the agent's behavior (e.g., "summarize in 50 words," "output only the final answer").
 - Format: The desired output structure (e.g., JSON, bulleted list, TSV, specific template with placeholders).
 - Examples (Few-Shot Prompting): Providing input-output pairs to guide the AI on the desired style or format. (e.g., for text classification: "Text: The weather is fine. Sentiment: Neutral").
 - Output Indicator: A phrase at the end of the prompt to prime the model for a specific output form (e.g., "Here's a bulleted list of key points:\n- ").
- **Popular Frameworks for Structured Prompting:**
 - COSTAR (Context, Objective, Style, Tone, Audience, Response): A highly structured and adaptable framework for comprehensive prompt design, treating prompt writing as a "full-stack design challenge."
 - CRAFT Framework (Context, Result, Explain, Audience, Tone, Edit): Useful for creating diverse output types tailored to specific audiences.
 - RACE Framework (Role, Action, Context, Execute): Helps in crafting focused and strategic prompts, often used in marketing scenarios.
 - Template Pattern: Providing a template with placeholders for the LLM to fill in, ensuring consistent formatting for outputs it wouldn't ordinarily use.

- **Benefits for Agentic Applications:**

- Improved Clarity and Consistency: Ensures outputs follow a predefined structure, making them easier to understand and compare.
- Reduced Off-Topic Responses and Hallucinations: Guides the LLM towards relevant information and minimizes irrelevant tangents or factual errors by limiting its "playfield."
- Enhanced Scalability and Efficiency: Streamlines prompt design, tuning, and iteration, making the process more efficient and adaptable across diverse tasks.
- Better Integration with Business Workflows: Enables AI models to operate reliably within specific business goals, regulatory environments, and multi-lingual scenarios.
- Facilitates Tool Use: More detailed and structured prompts clearly define tool access, purpose, input/output schemas, and how the agent should interpret tool outputs.

4. Practical Prompting Pro-Tips: Dos and Don'ts for Agentic Success Master the art of crafting prompts that lead to predictable and effective agent behavior.

- **DOs for Agentic Prompts:**

- Be Clear, Specific, and Concise: Every word matters. Provide direct instructions and avoid ambiguity.
- Provide Ample Context: Assume the agent knows nothing beyond what you explicitly tell it. Include all necessary background information.
- Assign a Clear Role/Persona: Define the agent's identity (e.g., "You are a senior data analyst") to guide its tone, vocabulary, and focus.
- Use Positive Instructions: Tell the agent what to do, rather than what to avoid. For example, "Output only the final answer" instead of "Don't explain the answer."
- Specify Desired Output Format and Tone: Clearly define the structure (e.g., JSON, bullet points, Markdown) and emotional register (e.g., professional, friendly) for the response.
- Provide Examples (Few-Shot Prompting): Illustrate the desired input/output behavior with a few well-chosen examples to guide the agent.
- Define Clear Goals and Constraints: Set limits on response length, scope, and specific parameters (e.g., "Summarize in 50 words," "Use Java 8 only").
- Use Delimiters for Clarity: For complex prompts, use symbols (like ### or ---) to clearly separate different sections of instructions or context.
- Encourage Planning and Reflection: Instruct the agent to "think step by step" or review its own output for self-correction before providing a final response.

- **DON'Ts for Agentic Prompts:**

- Don't Be Vague or Open-Ended: Avoid prompts like "Write an article" without specifying audience, purpose, tone, or topic.
- Don't Overload Prompts with Multiple Tasks: Break down complex requests into a series of focused, sequential prompts.
- Don't Use Conflicting Terms: Avoid contradictory instructions like "Write a detailed summary," which can confuse the model.

- Don't Include Changing State in System Prompts/Tool Definitions: If information (like current time) changes during a session, update it in subsequent user messages to avoid invalidating prompt cache.
- Don't Over-Engineer Simple Tasks: Not every problem needs a complex agentic solution; sometimes a simple script or a direct prompt is more efficient.
- Don't Be Overly Polite: Phrases like "please" or "thank you" do not affect the AI's response quality. Get straight to the point.
- Don't Rely on Slang or Metaphors: Unless specifically desired for a persona, avoid informal language that can introduce ambiguity.

5. Context is King: Mastering Dynamic Information Flow for Agents One of the biggest hurdles for agents is managing and recalling relevant information over time. We'll dive into "context engineering" – a critical discipline for robust agents.

- Retrieval-Augmented Generation (RAG): How to integrate external knowledge bases to provide agents with up-to-date, specific context, overcoming LLM context window limitations and reducing hallucinations.
- Memory Management Strategies: Techniques for maintaining conversational history and relevant past interactions without overwhelming the LLM's context window.
- Just-in-Time Context Provisioning: Dynamically feeding only the most relevant information to the agent at each step, optimizing performance and cost.

6. Production Readiness: Securing & Sustaining Your AI Agents Moving beyond development, we'll tackle the critical aspects of deploying and maintaining agentic applications in the real world.

- Combating Prompt Brittleness: Implementing rigorous testing and evaluation frameworks to ensure your prompts are robust across diverse inputs and model updates.
- Mitigating Prompt Injection Attacks: Design patterns and security considerations to protect your agents from malicious inputs, especially when they have access to external tools or sensitive data.
- Iterative Refinement & Monitoring: Establishing a feedback loop for continuous improvement, understanding that prompt engineering is an ongoing process of testing, evaluating, and refining for sustained performance in production.
- **Weaknesses and Constraints:**
 - While these strategies are powerful, they require significant engineering effort and a shift in mindset from simple prompting. They might not be necessary or cost-effective for extremely simple, single-turn LLM applications. Moreover, the effectiveness can still be somewhat model-dependent, requiring adaptation as LLMs evolve. Advanced context management and tool use can also introduce increased operational complexity and API costs.